



ATP: Achieving Throughput Peak for DNN Training via Smart GPU Memory Management

WEIDUO CHEN, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China

XIAOSHE DONG, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China

FAN ZHANG, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China

BOWEN LI, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China

YUFEI WANG, Information and Telecommunications Branch, State Grid Shaanxi Electric Power Company, Xi'an, China

QIANG WANG*, Xi'an Jiaotong University, Xi'an, China

Due to the limited GPU memory, the performance of large DNNs training is constrained by the unscalable batch size. Existing researches partially address the issue of GPU memory limit through tensor recomputation and swapping, but overlook the exploration of optimal performance.

In response, we propose ATP, a recomputation and swapping based GPU memory management framework that aims to maximize training performance by breaking GPU memory constraints. ATP utilizes a throughput model we proposed to evaluate the theoretical peak performance achievable by DNN training on GPU, and provide the optimum memory size required for recomputation and swapping. We optimize the mechanisms for GPU memory pool and CUDA stream control, employs an optimization method to search for specific tensors requiring recomputation and swapping, thereby bringing the actual DNN training performance on ATP closer to theoretical values. Evaluations with different types of large DNN models indicate that ATP achieve throughput improvements ranging from 1.14~1.49×, while support model training exceeding the GPU memory limit by up to 9.2×.

CCS Concepts: • **Computing methodologies** → *Heuristic function construction*; • **Information systems** → *Storage management*.

Additional Key Words and Phrases: DNN training, GPU memory management, Training throughput, Performance improvement

1 Introduction

Deep Neural Networks (DNNs) have been widely applied in fields such as image recognition [30], object detection [18], and natural language processing [9, 27]. Throughout the development of DNNs, large models achieve tremendous success in various domains, such as ResNet, and GPT, among others. The scale of models and datasets has become crucial for ensuring the accuracy and generalization capabilities of DNNs. With its powerful matrix computation capabilities, the GPU has become the most popular deep learning accelerator. However, the limited GPU memory severely impacts the deployment and training performance of large DNN models. For instance, a

Authors' Contact Information: Weiduo Chen, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, Shaanxi, China; e-mail: 867488982@qq.com; Xiaoshe Dong, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, Shaanxi, China; e-mail: xsdong@xjtu.edu.cn; Fan Zhang, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, Shaanxi, China; e-mail: zhangfan2022@stu.xjtu.edu.cn; Bowen Li, School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, Shaanxi, China; e-mail: lbwin520@stu.xjtu.edu.cn; Yufei Wang, Information and Telecommunications Branch, State Grid Shaanxi Electric Power Company, Xi'an, Shaanxi, China; e-mail: wyfzidane@stu.xjtu.edu.cn; Qiang Wang, Xi'an Jiaotong University, Xi'an, Shaanxi, China; e-mail: wangqiang1989@xjtu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2024 Copyright held by the owner/author(s).

ACM 1544-3973/2024/11-ART

<https://doi.org/10.1145/3701996>

GPT model with 12 Transformer modules and 117 million parameters can only train 12 batches on a GPU with 32GB memory [27]. Moreover, commonly used virtualization techniques in cloud computing services segment GPU resources [17, 34], making memory limit even more stringent. The memory constraints increase the difficulty of deploying models on GPUs and restrict batch sizes during training, thereby preventing the full utilization of GPU computing power.

To address the issue of GPU memory constraints, methods such as tensor compression [6], mixed precision [21] have been proposed to decrease the memory requirement of DNN training. However, evaluating the impact of these methods on model accuracy is challenging and often needs to be determined through experiments.

Other approaches utilize the data access patterns during training, employing memory swapping to break the constraints of GPU memory. In addition to the excessive model parameters, the practice of existing DNN frameworks [1, 15, 25] storing all parameters in GPU memory during training is also a significant cause of GPU memory shortages. Since training proceeds layer by layer, only the data related to the current layer needs to be accessed during each layer's computation, leaving most of the data in GPU memory idle most of the time. Consequently, data swapping have been proposed, utilizing host memory as the extra memory of GPU [16, 19, 20, 35]. They use data prefetching based on learning the patterns of page table changes to optimize the performance of data swapping. The relatively fixed computation and memory access patterns of DNN training enable researchers to propose swapping methods specifically for DNN training. These studies consider tensors as the fundamental objects for swapping and devise swapping strategies based on the size of tensors and the computation graph of DNNs [5, 13, 28]. Tensor recomputation is also a category of methods tailored for DNN training, referring to delete unused tensors in forward propagation and regenerating them in backward propagation [3, 14]. Some researches combine both recomputation and swapping to achieve higher performance [11, 26, 32].

Existing research generally considers the search of tensor management solution as the primary issue, which primarily includes the selection of recomputation and swapping tensors. These methods focus on the overlap of communication and computation during training, as well as the time cost of recomputation. They search for optimal tensor management solutions through expert knowledge [28], greedy strategies [36], heuristic algorithms [11, 13], and similar approaches.

However, the main objective of breaking GPU memory constraints is to enhance performance, and focusing solely on the selection of recomputation and tensor swapping does not sufficiently improve performance. Training performance can be expressed in terms of throughput, which is defined as the number of samples processed per unit of time. The factors that influence this include computation time, batch size, and the memory size used for recomputation and swapping. Studying the relationship between these factors and throughput is crucial, as it enables us to uncover the maximum achievable performance and develop training methods that realize this potential.

In response to the aforementioned issue, we propose ATP, a GPU memory management framework aimed at maximizing throughput for DNN training, which explores the achievable optimal performance of DNN training by breaking GPU memory limitations. ATP comprehensively considers the impact of batch size, recomputation and swapping memory sizes on throughput by a throughput model, searching for the optimal configuration of DNN training from a performance perspective. In ATP, the optimal training configuration encompasses the batch size, memory sizes for recomputation and swapping, and the operation of each tensor in the DNN.

To achieve the research objectives, ATP's design should address two key challenges. Firstly, increasing the batch size of DNN training contributes to the full utilization of GPU computing cores but may lead to the increased time costs for recomputation and swapping. Therefore, ATP needs a mathematical model to describe the relationship between tensor management solutions and throughput, and a throughput peak search strategy. Secondly, for the memory management framework, it should ensure the full utilization of GPU memory and minimize the additional time cost incurred by recomputation and swapping. On the one hand, swapping out and deleting

tensors may result in memory fragmentation, and ATP's memory management mechanism should aim to reduce the cost of fragment rearrangement as much as possible. On the other hand, the layers' computations and tensor swapping are generally deployed on two different CUDA streams for parallel execution. ATP strives to avoid potential synchronization overhead on both streams.

This paper claims the following contributions:

- We design and implement ATP, which allows users to explore the peak performance achievable in DNN training with excess GPU memory usage.
- We propose a throughput model, which is used to search for the theoretical throughput peak and allocate theoretical optimal recomputation and swapping memory size.
- To make ATP's actual performance close to the theoretical throughput peak, we optimize the GPU memory pool and CUDA stream control mechanism on one hand, and propose tensor generation efficiency metrics and genetic algorithm-based tensor management solution search algorithm on the other hand.
- We evaluate ATP using three types of DNNs: convolutional network, recurrent network, and transformer. Experimental results demonstrate that ATP is applicable to various types of DNNs, achieving a maximum performance improvement of 1.49× and supporting training up to 9.2× beyond the GPU memory limit.

2 Background

2.1 The Impact of Batch Size on Training Throughput and GPU Memory Requirement

In DNN training, increasing batch size is able to achieve higher throughput to some extent. A significant reason of the throughput improvement is the increase in computational intensity [33] that comes with a larger batch size. Computational intensity is defined as the number of floating-point operations performed per bit of memory accessed. Increasing computational intensity means that the same memory access can accommodate more computations, which helps reduce the proportion of memory access time and allows for better overlap of computation and memory access times in the processor. In typical DNN models, the majority of computational demand comes from convolution (CONV) and general matrix multiplication (GEMM). The theoretical computational intensities of CONV and GEMM, can be approximately represented by Table 1, where b denotes the batch size. It is noted that the denominators of I_{GEMM} and I_{CONV} both contain constant terms that are independent of b . Therefore, increasing the batch size will cause I_{GEMM} and I_{CONV} to increase. While increasing performance, enlarging the batch size also raises the demand for GPU memory. Equation 1 reflects the theoretical memory requirement $M(b)$ for training a L model with L layers [23, 29], where x , dx , w , and dw represent the output tensor, the gradient of the output tensor, the weights, and the gradient of the weights for each layer, respectively. *ReserveSpace* refers to the space used in recurrent layers and attention layers to store intermediate results, which must be retained for use in backward propagation. *WorkSpace* represents a shared additional space required by some layers' computation. The terms with b as a coefficient will increase as the batch size increases, which make some larger models to be trained only with small batch sizes under the limitation of GPU memory.

Table 1. Computational intensities [4]

Operation	Dimensions of the associated tensors	Computational intensity
GEMM	Input: $X \in \mathbb{R}^{b \times d}$, Weight: $W \in \mathbb{R}^{d \times k}$, Output: $Y \in \mathbb{R}^{b \times k}$	$I_{GEMM} = \frac{bk(2d-1)}{4b(d+k) + 4dk}$
CONV	Input: $X \in \mathbb{R}^{b \times c \times h \times w}$, Filter: $K \in \mathbb{R}^{c_o \times c \times r \times s}$, Output: $Y \in \mathbb{R}^{b \times c_o \times h_o \times w_o}$	$I_{CONV} = \frac{bc_o h_o w_o (2crs-1)}{4b(chw + c_o h_o w_o) + 4c_o crs}$

$$M(b) = b \sum_{l=1}^L (size(x_l) + size(dx_l) + ReserveSpace(l)) + \sum_{l=1}^L (size(w_l) + size(dw_l)) + WorkSpace(b) \quad (1)$$

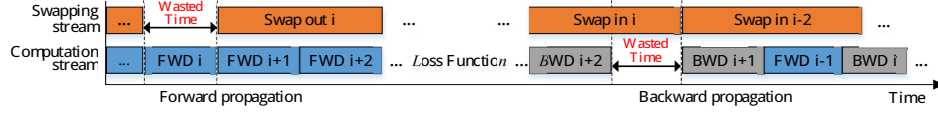


Fig. 1. The timeline of DNN training with tensor swapping and recomputation, where $FWD i$ denotes the forward computation of the i th layer, $BWD i$ represents the backward computation of the i th layer, $Swap out i$ and $Swap in i$ respectively indicate the swapping out and swapping in of tensors generated by the i th layer.

2.2 Swapping and Recomputation

Swapping and recomputation enable the GPU to train models that exceed memory limits. Swapping involves two operations: swapping out and swapping in. Swapping out refers to copying tensors' data from GPU memory to CPU memory in forward propagation, freeing up the occupied GPU memory space. Swapping in fetches the tensors' data back from the CPU memory back to GPU memory in backward propagation before. Computation and swapping generally run on separate CUDA streams that can execute in parallel [11, 13, 28]. During the training process, the order in which tensors are generated and used is determined by the computational graph of the DNN model. Inappropriate tensor swapping can lead to unnecessary synchronization time, as illustrated in Fig. 1. There are two types of synchronization shown in the figure, $Swap out i$ requires waiting for $FWD i$ to generate tensors, and the synchronization overhead between $BWD i+2$ and $BWD i+1$ is because $BWD i+1$ needs to wait for $Swap in i$ to retrieve tensor i . The essence of swapping is to exchange GPU memory resources for PCIe bandwidth resources. Exceeding the total swapping volume that PCIe interface can transfer within the forward or backward propagation time will inevitably result in synchronization time.

Recomputation methods reduces memory consumption by deleting some output tensors of layers during the forward propagation. Since these tensors might be needed for gradient computation, the associated layers' forward computations must be re-executed during the backward propagation to regenerate them [14, 32]. As shown in Fig. 1, $BWD i$ needs to wait for $FWD i-1$ to regenerate the required tensors.

There are two key differences between recomputation and swapping: recomputation only applies to tensors generated during forward propagation, not all tensors, and swapping time can overlap with computation time, while recomputation inevitably extends the backward propagation time.

3 Observation and Motivation

3.1 Limitations of GPU Memory Management Methods

We compared existing memory management approaches from six perspectives as shown in Table 2. The existing works like vDNN [28], moDNN [5], SwapAdviser [13], and Checkmate [14] solely employ swapping (SW) or recomputation (RE) to reduce memory usage. However, a more effective training performance can be achieved by selectively using recomputation or swapping based on the computation time and size of tensors. SuperNeurons and Capuchin employ both recomputation and swapping; however, their tensor management solution search methods are not globally optimized. SuperNeurons chooses specific types of tensors for recomputation or swapping, such as recomputing activation and normalization layer feature maps and swapping convolutional layer feature maps [32]. Capuchin uses a greedy algorithm and its proposed memory saving per second (MSPS) metric (Equation 2) to determine the recomputation tensors [26]. The search for a tensor management solution can be seen as an optimization problem aiming to minimize model iteration time while selecting tensors for recomputation and swapping. HOME utilizes particle swarm optimization (PSO) to solve this optimization problem, but it does not recompute or swap tensors other than feature maps [11]. Although the training of CNNs incurs major memory costs from feature maps, in RNNs and transformers, feature maps account for only a small portion of the total memory requirement. Therefore, ATP is designed as a tensor-granularity GPU memory management framework using a global optimization approach.

Table 2. Comparison with other GPU memory management methods

	RE	SW	Management objective	Global Optimization	Search method	Throughput peak search and allocation of recomputation and swapping sizes
vDNN	×	✓	Feature map	×	Qualitative analysis	×
moDNN	×	✓	Feature map	×	Expert knowledge	×
SwapAdvisor	×	✓	Tensor	✓	Genetic algorithm	×
Checkmate	✓	×	Feature map	✓	MILP	×
SuperNeurons	✓	✓	Feature map	×	Qualitative analysis	×
Capuchin	✓	✓	Feature map	×	Greedy algorithm	×
HOME	✓	✓	Feature map	✓	PSO	×
ATP	✓	✓	Tensor	✓	Genetic algorithm	✓

$$MSPS = \frac{MemorySaving}{RecomputationTime} \quad (2)$$

The above works only focus on how to choose tensors for swapping and recomputation, without proposing an optimized solution for performance. Although the theoretical computational intensity increases with larger batch sizes, the actual training throughput will not continue to increase due to the limitations of GPU computational resources and memory. While recomputation and swapping can break through memory limitations, they may also introduce additional time overhead. Take swapping as an example, although this operation runs in parallel with computation, if the necessary data transfer cannot be completed within the computation time, the model's iteration time will be extended. In Fig. 2a, we estimated the potential changes in training throughput when using the swapping method based on a simple principle (Equation 3). Specifically, the model iteration time is determined by the maximum of the computation and swapping times, while the estimated throughput is the ratio of the number of samples to this time (for cases where the training exceeds the memory limits, we sum up all the forward and backward computation times of all layers as the iteration time). It can be observed that the estimated throughput initially increases and continues to grow even after the model size exceeds the memory limit. This is because the required memory for swapping is relatively small and can be completed within the model's computation time. However, as the batch size continues to increase to a certain point, the throughput begins to consistently decline. This trend is determined by the changes in model size and computational performance. Since the model size increases linearly, the swapping time also increases linearly once the memory limit is exceeded. However, the rate of performance improvement slows down because the trend of increasing computational intensity is diminishing and the GPU's computational resources are limited. Fig. 2b, c and d show the recomputation and swapping times for parameters of different layers, as well as the proportion of these parameters in the model. Notably, some layers have a large proportion of parameters but very short computation times, such as batch normalization (BN) and layer normalization (LN) layers. By regenerating their parameters through recomputation when the memory size to be swapped is excessive, we can reduce performance losses caused by swapping. This could potentially further increase the throughput peak shown in Fig. 2a. In summary, designing a throughput prediction model that encompasses both swapping and recomputation, along with a method to search for the throughput peak, is valuable. It can help us determine the optimal amount of additional memory to use during model training to achieve the optimum performance.

$$EstimatedThroughput = BatchSize \cdot \left(\max \left(ComputationTime, \frac{SwappingSize}{Bandwidth} \right) \right)^{-1} \quad (3)$$

3.2 Deficiencies in GPU Memory Management Framework Design

CUDA Stream Control Mechanism: In the CPU+GPU architecture, the computation of model training is predominantly handled by the GPU, with the CPU tasked with launching CUDA kernel according to the order defined by the computational graph. Each kernel corresponds to a specific operation, primarily involving layers'

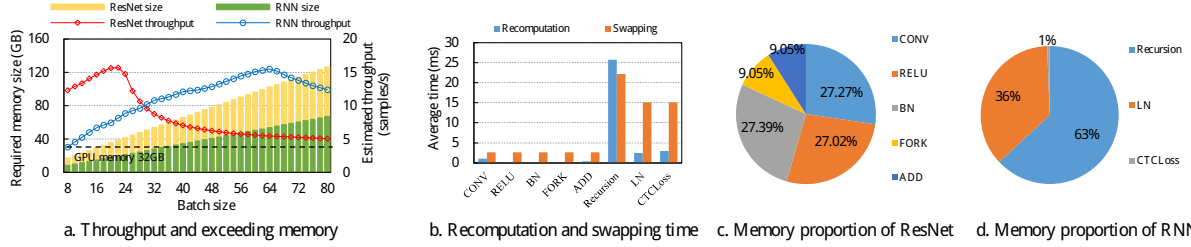


Fig. 2. Observations of training performance and memory requirement. The experimental device is a Tesla V100 GPU with 32GB of memory. The workload includes a 1022-layer ResNet with an input size of $3 \times 224 \times 224$ and a 64-layer RNN model with a sequence length of 512.

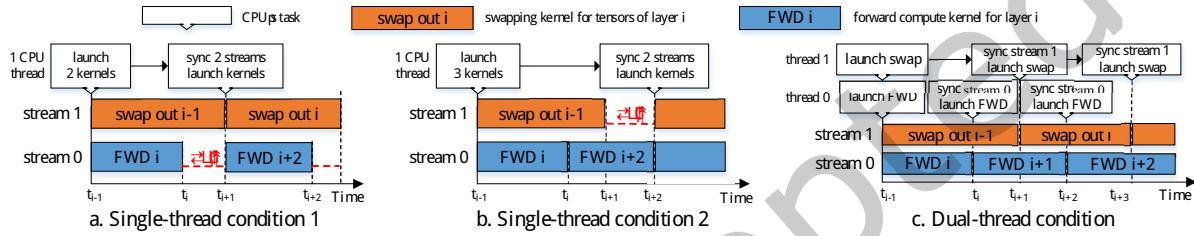


Fig. 3. The operation of CUDA kernels under single-thread [26] and dual-thread CPU control

computation and tensor swapping on PCIe bus. CUDA allows computation and swapping kernels to run on two separate streams, enabling the overlap of computation and tensor swapping operations. The control of the CPU over kernels on CUDA stream requires calling the launch and the stream synchronization commands. After a kernel is launched, it immediately returns, allowing the CPU to perform subsequent tasks, while the launched kernel continues to run on the CUDA stream until completion. It is necessary for the CPU to use the synchronization command of the corresponding stream to determine the end time of the launched kernels in order to assess the status of related tensors, such as whether they have been generated or swapped. Stream synchronization is a blocking operation, meaning that the CPU needs to wait for the launched kernel to complete. Current frameworks use a single CPU thread to launch both computation and swapping kernels, which may lead to some time wastage. Fig. 3a is a coupled kernel launch approach, where the CPU initiates both the compute kernel and swapping kernel for a layer, followed by synchronization of the two streams [28]. If the computation time is shorter, the next computation kernel has to wait until the swapping kernel is completed before it can start (and if the swapping time is shorter, there will also be waiting time). Launching multiple kernels at once on a single stream does not resolve the issue [26], as any differences in runtime between tasks on two streams will result in waiting times (Fig. 3b). A dual-thread control mechanism, as illustrated in Fig. 3c, can address this problem effectively. By using two separate CPU threads to initiate kernels on different streams, it is possible to avoid the effects of time differences between computation and swapping kernels.

GPU Memory Pool: When GPU memory is sufficient, each tensor can occupy fixed memory spaces and the memory utilization will not change during training. Recomputation and swapping can lead to fragmentation issues in GPU memory due to the continuous generation and removal of tensors of different sizes. When tensors need to be swapped in or recomputed, a simple way is to use "cudaFree()" to release some unused tensors, and then use "cudaMalloc()" to reallocate memory for them [36]. But CUDA APIs are quite time-consuming. To improve performance, some works design memory pools to manage the memory space required for tensors [5, 13, 25, 32]. Memory pools typically requests a large block of GPU memory and then divides it into many smaller blocks for management [13, 25]. The number of blocks each tensor needs is determined based on its size. When a tensor is

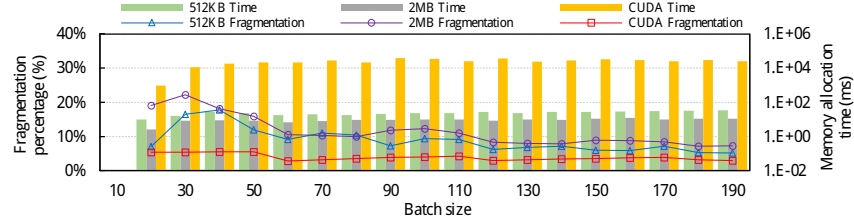


Fig. 4. GPU memory fragmentation and memory allocation time of CUDA APIs and memory pools with the block sizes of 2MB and 512KB. The workload is a 1022-layer ResNet with the input size of $3 \times 224 \times 224$.

removed, the memory is not freed; instead, the blocks occupied by the tensor are marked as "free" for use by other tensors. We tested the average proportion of memory fragmentation in GPU memory and the memory allocation time for CUDA APIs and memory pools with two different block sizes (2MB and 512KB) as shown in Fig. 4. CUDA has less fragmentation than memory pools because it can map non-contiguous physical memory blocks to a contiguous virtual address space through page tables, while memory pools generate many small, non-contiguous blocks due to frequent tensor swapping and recomputation. However, compared to the updating and remapping of page tables, the time overhead of memory pools is much lower. The time overhead of memory allocation must be avoided, as adding unnecessary time contradicts the goal of improving throughput. It's noted that fragmentation occurs only in the address segments where tensors requiring recomputation and swapping are located. Therefore, an efficient strategy would be to limit the addresses of these tensors to a small area rather than distributing them across the entire GPU memory range. We propose a multi-buffer memory pool in ATP, which consists of a few memory buffer and allows only tensors that require recomputation and swapping to utilize these blocks in a time-multiplexed manner. This approach can eliminate the time spent on memory management, as well as limit fragmentation size to some extent.

4 Design

ATP is designed to explore how DNN models can achieve maximum performance by breaking the GPU memory limit. To achieve this goal, ATP needs to have a fast throughput peak search capability. We propose a throughput model under memory constraints, which includes only a few formulas and requires minimal information to estimate training throughput. ATP first utilizes this throughput model to search for the throughput peak and the corresponding training configuration, which includes the batch size, total recomputation and swapping size. Secondly, ATP should strive to reduce the synchronization time caused by recomputation and swapping. We optimize the CUDA stream control mechanism and GPU memory pool, and use a multi-step optimization approach to separately search for recomputation and swapping tensors based on the training configuration.

This section first introduces the overall architecture of ATP to clarify the relationships among its various components. Then, we discuss how to use the throughput model for throughput peak search and describe the design of ATP's stream control and GPU memory pool. The recomputation tensor search and the swapping tensor search are introduced last because their designs are influenced by the memory pool.

4.1 Overview of ATP Framework

The overall framework of ATP, as depicted in Fig. 5, consists of two Stages and five components. Stage 1 encompasses an information collector and a throughput model. According to the DNN model information and PCIe bandwidth, it seeks the peak of theoretical throughput, determining the corresponding batch size b and determining the minimal recomputation memory size M_R and swapping memory size M_S for training a DNN model that exceeds GPU memory limit. Theoretical throughput represents the upper bound of throughput when training a model on a GPU with a given batch size. Any swapping and recomputation strategies can only bring the

actual training throughput close to this upper bound, but not exceed it. In Stage 2, the specific recomputation and swapping tensors are searched based on the b , M_R and M_S obtained in Stage 1, to make the actual throughput as close as possible to the theoretical throughput. We posit that if the theoretical throughput identified in Stage 1 is maximized, the actual throughput is highly likely to also reach its maximum value. Specifically, the search result should minimize the time of each training iteration while satisfying the actual memory size of recomputing and swapping is not less than M_R and M_S . Finally Stage 2 furnishes sets of recomputation tensor set R and swapping tensor set S to the training controller. This controller incorporates CUDA stream control mechanism and GPU memory pool, tasked with executing the training of the DNN model.

Stage 1. Throughput Peak Search: First, the information collector traverses all layers and tensors of the DNN model, recording model information (Model Info (b)) for training with a batch size of b . This includes the forward and backward computation times for each layer, as well as the size of each tensor x . Throughput model estimates the theoretical throughput $\hat{T}(b)$ during training using Model Info (b) and PCIe bandwidth. $\hat{T}(b)$ represents the upper bound of throughput under memory limit. Let b_0 be the maximum batch size for normal training. In Stage 1, we start the search for the theoretical throughput peak $\hat{T}_{max}(b_1)$ from $b = b_0$ with a step size of Δb . According to the analysis in section 3.1, $\hat{T}(b)$ generally first increases and then decreases. We stop the search when $\hat{T}(b)$ decreases. However, to prevent multiple peaks caused by runtime errors, we set a window size of approximately 2 to 3 times Δb (Equation 4). If $\hat{T}(b)$ within the entire window does not exceed the historical maximum value of theoretical throughput, we consider that $\hat{T}_{max}(b_1)$ has been found. ATP utilizes the throughput model to allocate M_R and M_S for the training with the batch size of b_1 , as well as G , which is a sequence obtained by arranging all tensors in descending order of their generation efficiency. The details of the throughput model's design and operation are explained in section 4.2.

$$\hat{T}(b - \epsilon\Delta b), \dots, \hat{T}(b - \Delta b), \hat{T}(b) < \hat{T}_{max}(b_1) \quad (\epsilon = 2, 3) \quad (4)$$

Stage 2. Tensor Management Solution Search: The tensor management solution includes specific recomputation tensor set R and swapping tensor set S . As shown in the right part of Fig. 5. During the training process, the computation can cover the swapping process, while recomputation will definitely increase the computational time. Therefore, ATP first determines the set R and then determines S from the remaining tensors. We select the tensors from sequence G that are most suitable for recomputation and add them to R until the actual total recomputation size is not less than M_R . The search for swapping tensors is considered as an optimization problem with the shortest training time as the object and the actual total size of swapping not less than M_S as the constraint, and it is solved using genetic algorithms. ATP estimates the training time for each swapping solution generated by the genetic algorithm through the TimeEvaluator without actual training on the GPU, greatly improving the search speed.

Finally, based on the outputs of the preceding two stages, the training controller allocates GPU memory space for tensors in the model according to the batch sizes b and sets R and S , and initiates the training of DNN model on the GPU.

4.2 Throughput Model

This section introduces the throughput model and elaborates on how to use it to estimate the theoretical training throughput, as well as allocate the memory sizes for recomputation and swapping. The throughput model is represented by Equation 5, where $\hat{T}(b)$ and $\hat{t}(b)$ respectively represent theoretical throughput and training time (a single training iteration time) of the DNN training using a batch size of b . In the equation, $t_F(b)$ and $t_B(b)$ respectively indicate the pure computation time of forward and backward propagation without swapping or recomputation. They are obtained by accumulating the N layers' forward computation time $t_F(i, b)$ and backward computation time $t_B(i, b)$. Since swapping and tensor swapping are performed in parallel, the theoretical forward

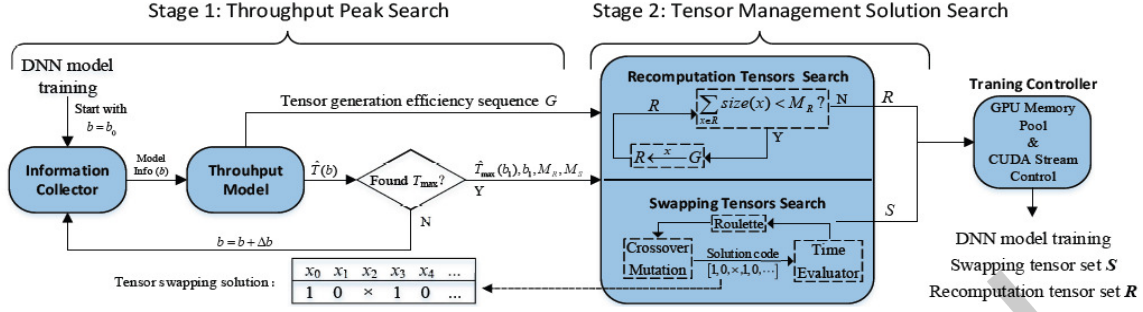


Fig. 5. The overall architecture of ATP

propagation time is the maximum of $t_F(b)$ and the total swapping time, and the theoretical backward propagation time follows the same principle, but adding the recomputation time. Since the tensors swapped out during the forward propagation typically need to be fully retrieved during the backward propagation, the total time of swapping out and swapping in is theoretically the same and can be worked out based on the PCIe bandwidth B and M_S . $M(b)$ represents the memory requirement for training, and M_E denotes the part exceeding the GPU memory (M_{GPU}). This part is divided into M_R and M_S , where the actual total size of tensors in R and S must not be smaller than M_E ; otherwise, the GPU memory cannot meet the training demands. The last inequality in Equation 5 reflects the constraint of training with excess GPU memory usage, which means it is essential to ensure that the relevant tensors of any layer in the DNN model can be allocated to M_{GPU} . Otherwise, there will inevitably be a shortage of memory during the training process.

$$\begin{cases}
 \hat{T}(b) = b \cdot (\hat{t}(b))^{-1} \\
 \hat{t}(b) = \max\left(t_F(b), \frac{M_S}{B}\right) + \max\left(t_B(b) + \sum_{x \in R} t_F(x, b), \frac{M_S}{B}\right) \\
 t_F(b) = \sum_{i \in [1, N]} t_F(i, b), \quad t_B(b) = \sum_{i \in [1, N]} t_B(i, b) \\
 M_E = M_{GPU} - M(b) = M_R + M_S < \sum_{x \in R} \text{size}(x) + \sum_{x \in S} \text{size}(x) \\
 \max_{\text{layer} \in \text{DNN model}} \left(\sum_{x \in \text{layer}} \text{size}(x) \right) \leq M_{GPU}
 \end{cases} \quad (5)$$

The Allocation for M_R and M_S can be divided into two scenarios: when M_E is small, the excess memory can be entirely managed through swapping, while for larger M_E values, it requires incorporating recomputation.

Scenario 1: When M_E satisfies $M_E B^{-1} < t_F(b)$ (due to the larger computation load of the backward propagation, typically $t_F(b) < t_B(b)$), the theoretical computation time can completely overlap with the total time for swapping, thus only swapping is used, as shown in Equation. 6.

$$\hat{t}(b) = t_F(b) + t_B(b), \quad (M_S = M_E, M_R = 0) \quad (6)$$

Scenario 2: When $M_E B^{-1} > t_F(b)$, if M_E is fully allocated for swapping, there will definitely be non-overlapping swapping time. In this case, ATP's strategy is to initially allocate the memory M_{S1} that might complete swapping within $t_F(b)$, and the remaining $M_E - M_{S1}$ is then allocated to M_R and M_{S2} . At this point, $\hat{t}(b)$ can be obtained from Equation 7. Since the throughput model estimates the maximum throughput, M_R and M_{S2} should consist of the tensors with the maximum recomputation and swapping efficiency. We propose tensor generation efficiency $g(x)$, which reflects the fastest generation method and generation efficiency for a tensor. As shown in Equation 8, the recomputation efficiency is the tensor size recomputed per unit time and the swapping efficiency is equivalent to the bandwidth. Algorithm 1 demonstrates the allocation process for M_R and M_S . First, the tensors in the DNN

model are arranged in descending order based on $g(x)$ to generate sequence G (line 2). Then the tensors from G are sequentially selected and their size are add to M_R and M_S according to their suitable generation methods, until M_E is fully allocated (lines 3~10). This allocation of M_R and M_S is the most efficient, and the actual recomputation and swapping size should be as close to M_R and M_S as possible.

$$\begin{cases} M_{S1} = Bt_F(b), M_E - M_{S1} = M_{S2} + M_R \\ \hat{i}(b) = t_F(b) + \frac{M_{S2}}{B} + \max\left(t_B(b) + \sum_{x \in R} t_F(x, b), \frac{M_{S1} + M_{S2}}{B}\right) \end{cases} \quad (7)$$

$$g(x) = \max\left(\frac{\text{size}(x)}{t_F(x, b)}, B\right) \quad (8)$$

Algorithm 1: Allocation for M_R and M_S

Input : Model Info (b)
Output: M_R, M_S

```

1  $M_{S1} \leftarrow Bt_F(b), M_R \leftarrow 0, M_{S2} \leftarrow 0;$ 
2 Sort all the  $x$  of DNN model in descending order based on  $g(s)$ , get  $G = \{x_i | x_i \in \text{DNN model}, g(x_i) > g(x_{i+1})\};$ 
3 while  $M_R + M_{S2} < M_E - M_{S1}$  do
4   Get next  $x$  from  $G$ ;
5   if  $g(x) = \text{size}(x)/t_F(x, b)$  then
6      $M_R \leftarrow M_R + \text{size}(x);$ 
7   else
8      $M_{S2} \leftarrow M_{S2} + \text{size}(x); \quad // \quad g(x) = B$ 
9   end
10 end
11  $M_S = M_{S1} + M_{S2};$ 

```

4.3 Multi-buffer Memory Pool

The design goal of multi-buffer memory pool is to confine any potential fragmentation to a small range of GPU memory, while avoiding the introduction of additional time for defragmentation. ATP adopts the memory allocation scheme shown in Fig. 6. Considering that fragmentation only occurs with swapping and recomputation tensors, a very small portion of memory is designated as a memory pool to temporarily store these tensors, while the majority of the memory is allocated to other tensors. The memory pool is constructed based on multi-buffer technology. The multi-buffer mechanism is commonly used in processor design [24], high-performance computing, and storage[31] to coordinate the speed differences between computation, memory access, and communication processes. In ATP, configuring multiple buffers helps coordinate the computation and swapping processes. When a swapped tensor needs to occupy a buffer for an extended period, such as in a residual structure where the tensor is required for subsequent addition layers and cannot be released immediately after its data is copied to CPU memory, other buffers can be used to store new tensors. Once it is swapped out and no longer needed, the buffer it occupies will be marked as free for use by other tensors. The size of each buffer does not need to be very large— just large enough to accommodate any single tensor. Therefore, the size of the entire memory pool is limited to the size range of a few tensors. The configuration of the number of buffers is related to the structure and computational logic of the model.

Swapping Buffer: The number of swapping buffers impacts the flexibility of tensor selection for swapping. Taking the example of the linear model illustrated in Fig. 7a, in the computational process of a particular layer, a single buffer can only accommodate storing the inputs, outputs, or weights of that layer. However, with three buffers, it becomes possible to simultaneously store three tensors: inputs, outputs, and weights. This indicates that in a linear network model, three buffers can adapt to any tensor swapping solution. In the case of a branching

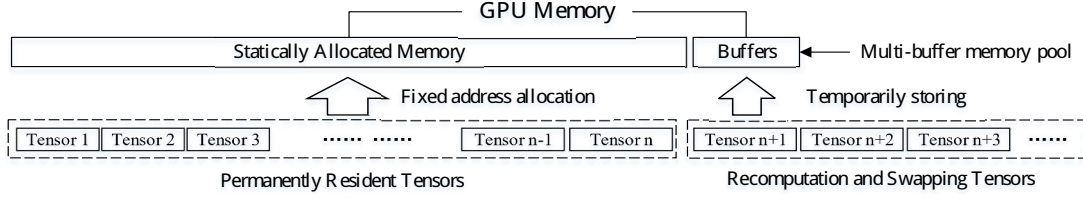


Fig. 6. GPU memory allocation of ATP

structure, the number of buffers is related to the number of branches. For instance, as shown in Fig. 7b, a structure such as *FWD 4* with three branches would require four swapping buffers to accommodate all tensors.

Recomputation buffer: Unlike swapping, there is data dependency among tensors involved in recomputation. In cases where several consecutive layers' tensors require recomputation, these feature maps need to be computed in sequence according to the forward propagation order. Recomputed tensors not only occupy recomputation buffers during gradient calculation but also during the recomputation process. In the scenario depicted in Fig. 8a with a single buffer, it is not possible to select adjacent layer feature maps for recomputation because a single buffer cannot simultaneously store the input and output of forward propagation. If consecutive feature maps require recomputation, more buffers are necessary. Fig. 8b demonstrates the continuous recomputation process where the recomputation of x_i must start from *FWD* $i - 2$ to *FWD* i since *FWD* $i - 2$, *FWD* $i - 1$, and *FWD* i 's computations are interdependent. During this period, x_{i-2} , x_{i-1} , and x_i must reside in the buffer until they are entirely utilized by the respective layers' backward computation. Therefore, the number of recomputation buffers determines the maximum consecutive recomputation count.

Since recomputation and swapping tensors are only allocated within the memory pool, the potential fragmentation size will not exceed the size of the memory pool. The fragmentation size of a buffer may generate depends on the size of the tensor it holds. Assuming the number of buffers for swapping and recomputation are denoted as *SBN* and *RBN* respectively, the maximum fragmentation size can be estimated using Eq. 9. According to the analysis above, only a small number of buffers (each buffer sized approximately for a larger tensor) are needed to meet the general requirements of most models. Considering that a model typically contains hundreds to thousands of tensors, the total size of the fragmentation will be far less than the model's memory requirements. Therefore, ATP does not need to consider dynamically merging such small amounts of fragmentation during the training process.

$$MaxFragmentSize = SBN \left(\max_{x \in S} size(x) - \min_{x \in S} size(x) \right) + RBN \left(\max_{x \in R} size(x) - \min_{x \in R} size(x) \right) \quad (9)$$

4.4 Dual-thread Control Mechanism for CUDA Stream

ATP uses two CPU threads, Thread 0 and Thread 1, to control the CUDA computation and communication streams, respectively, in order to avoid the issue described in Section 3.1 where the communication stream cannot execute continuously. Under the dual-thread control mechanism, there needs to be some necessary interaction between Thread 0 and Thread 1. For instance, before Thread 1 initiates the swapping out of a particular tensor, it

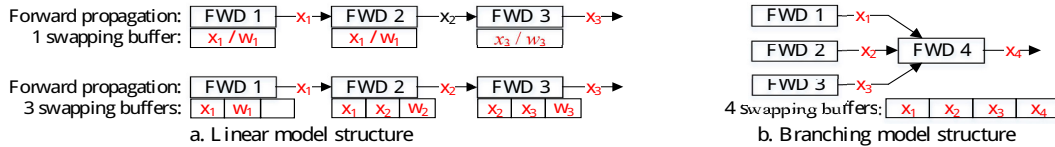


Fig. 7. Swapping buffers of GPU memory pool. *FWD* i represents the forward computation for the i th layer, x_i and w_i denote the output tensor and weight of the i th layer.

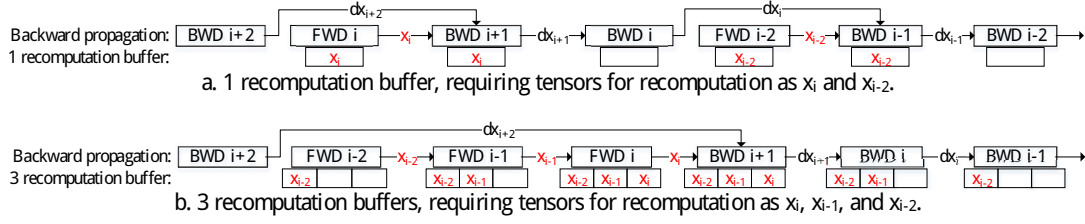


Fig. 8. Recombination buffers of GPU memory pool. $BWD i$ and $FWD i$ respectively represent the backward computation and recombination (forward computation) of the i th layer, x_i denotes the output tensor of the i th layer.

should verify whether that tensor has been generated by the computation stream. Similarly, before Thread 0 executes the backward computation of a layer, it needs to ensure that the required tensors have been swapped in by the communication stream. We design data structures for tensors and the buffers of the GPU memory pool. As shown in Table 3, both the tensor and buffer have several attributes. These attributes enable the two threads to determine the states of tensors and buffers, thereby deciding when to launch recombination and swapping in and out. If a tensor is set to a buffer, the tensor's *buffer* point to the buffer and the buffer's *tensor* point to this tensor.

Algorithm 2 illustrates the control logic of Thread 0. We sequentially launch the computation kernels of the layers according to the specified order in the computational graph (in the computational graph, nodes represent forward or backward computations of layers, while edges represent tensors outputted to the next layer). In the forward propagation, computational kernels are launched after each relevant tensor get a buffer (lines 5~7). And then, the states of relevant tensors are updated. When these tensors are swapped out by Thread 1, the occupied buffers become free (lines 9). Similarly, layers' computations start when the relevant tensors are all swapped in or recomputed in the backward propagation (lines 14~21). Recombination is implemented through a recursive function (lines 25~33), for consecutive recombination scenarios, we continuously access the previous layer of the current layer until an input tensor that does not require recombination.

The communication stream control is shown as Algorithm 3, where the swapping tensor set S is arranged in the order of their occurrence in the forward propagation. The swapping out of a *tensor* is launched when it is in the GPU memory (lines 2~3). And the related *buffer* can be free if the swapped *tensor* is useless (line 4). The order of swapping in is opposite to swapping out during the backward propagation (line 6). Once an available *buffer* exists, a tensor is started to swap into the *buffer*, and its state becomes *usefull* until it is fully utilized.

4.5 Recombination Tensors Search

ATP aims to select tensors with the highest recombination efficiency for recombination. We can sequentially select tensors from G that are more suitable for recombination. Constrained by the ATP's memory pool mechanism, when selecting tensors for recombination, it is essential to ensure that the number of tensors requiring consecutive recombination does not exceed the number of recombination buffers. Fortunately, layers containing tensors suitable for recombination and those suitable for swapping usually appear in pairs. For example, the feature map of a convolutional layer may have lower recombination efficiency, but that of the feature map generated by the adjacent activation layer or normalization layer is significantly greater. The method of recombination tensors search is to sequentially select tensors from G which is arranged according to recombination efficiency. If selecting a tensor results in too many consecutive recombination tensors, that tensor will not be selected.

Table 3. Data structures of tensor and buffer

Tensor	Attribute Values	<i>fcounter</i> 0, 1, 2, ...	<i>bcounter</i> 0, 1, 2, ...	<i>operation</i> RE, SW, KE	<i>state</i> <i>useful</i> , <i>useless</i>	<i>posistion</i> <i>cpu</i> , <i>gpu</i> , <i>nodata</i>	<i>buffer</i> buffer pointer
Buffer	Attribute Values	<i>gpu_address</i> data pointer	<i>state</i> swapping, ready	<i>type</i> RE, SW	<i>tensor</i> tensor pointer		

Algorithm 2: Thread 0: computation stream control

```

Input :CG; // The computational graph of the specified DNN model
1 foreach node of CG do
2   layer ← GetLayerInfo(node);
3   tensors ← GetTensors(layer); // Get the feature map, weight and other relevant tensors of layer.
4   if forward propagation then
5     if tensor.operation ≠ KE then Wait for free buffer, and set tensor into buffer; // buffer.tensor = NULL
6     if each relevant tensor has a buffer then
7       RunForwardKernel(layer), tensor.position ← gpu;
8       tensors.fcounter − −;
9       UpdateState(tensor); // if fcounter = 0 then tensor.state ← useless
10      if tensor.state = useless and tensor.operation = RE then tensor.position ← nodata;
11      node ← the next node of CG;
12    end
13  else
14    if tensor.position = cpu then Wait for tensors to be swapped in;
15    if tensor.position = nodata then Recompute(tensors);
16    if each relevant tensor.position = gpu then
17      RunBackwardKernel(layer);
18      tensors.bcounter − −;
19      UpdateState(tensor); // if bcounter = 0 then tensor.state ← useless, buffer.tensor ← NULL
20      node ← the next node of CG;
21    end
22  end
23 end
24 ResetAttributes(all the tensors in the DNN mode);
25 Function Recompute(tensor):
26   layer ← GetLayer(tensor); // Get the layer that generated this tensor.
27   last_tensor ← GetInput(layer);
28   if last_tensor.position = nodata then
29     Recompute(last_tensor, layer);
30   end
31   if tensor.position = cpu then Wait for last_tensor to be swapped in;
32   RunForwardKernel(layer), tensor.position ← gpu;
33   return;

```

Algorithm 3: Thread 1: communication stream control

```

Input :S = {tensor(1), tensor(2), ... tensor(|S|)}; // The ordered set of swapping tensors.
1 for i = 1 to |S| do
2   Wait for tensor(i).position = gpu;
3   SwapOut(tensor(i)), tensor(i).position ← cpu;
4   if tensor(i).state = useless then tensor(i).buffer.tensor ← NULL;
5 end
6 for i = |S| to 1 do
7   Wait for free buffer, and set tensor(i) into buffer;
8   SwapIn(tensor(i)), tensor(i).position ← gpu, tensor.state ← useful;
9 end

```

4.6 Swapping Tensors Search

As shown in Equation 10, the search for swapping tensor is modeled as an optimization problem with the goal of minimizing the iteration time of DNN model training, denoted as $t(b, ind)$, while satisfying the constraint that the actual swapping memory size m_S is not smaller than M_S . Here, the batch size b is determined in Stage 1, ind is the binary encoding of the tensor swapping solution, and $population$ is the set of ind . In ind , $s_i = 1$ or 0 indicates whether tensor x_i needs to be swapped or not (without considering tensors already identified). The TimeEvaluator estimates the iteration time $t(b, ind)$ by simulating the training process of the DNN model under the tensor swapping solution encoded by ind . In the search based on the genetic algorithm, the fitness function is

formulated as the weighted sum of the reciprocals of $t(b, ind)$ and the relative error between m_S and M_S . This function aims to steer the search for selecting the swapping solution with shorter iteration time and smaller excess swapped memory sizes. Based on our experience, the values of α and β are 1 and 5 respectively. Swapping Tensors Search stops after completing the specified number of iterations, and the final set of swapping tensors is determined by the historical best ind .

$$\begin{cases} \text{object: } \min_{ind} (t(b, ind)), t(b, ind) \approx \text{TimeEvaluator}(b, ind) \\ \text{subject to: } m_S \sum_{x_i \in R, s_i \in ind} s_i \text{size}(x_i) \geq MS, (s_i = 0 \text{ or } 1) \\ ind = \{s_i | i \in [1, m]\}, (m = \text{the number of tensors that do not belong to } R) \\ \text{population} = \{ind_i | i \in [1, PS]\}, (PS = \text{population size}) \\ \text{fitness}(ind) = \alpha t^{-1}(b, ind) + \beta \frac{m_S - M_S}{M_S} \end{cases} \quad (10)$$

The search process is illustrated in Algorithm 4. We initialize the *population* using a mutation operation (lines 1~4), flipping each bit in every all-zero individual in the population to 1 with a probability of i/PS . This approach ensures that the initial population covers scenarios ranging from no swapping to swapping all tensors. In each iteration, we first check whether each *ind* satisfies the constraint conditions and calculate its fitness f using TimeEvaluator, and the fitness of the *ind* which does not meet the constraint conditions is set to 0 (lines 5~14). Then, the historical maximum fitness f_{best} and the best solution ind_{best} are updated (lines 15~18). The update for the new *population* involves sampling, crossover, and mutation (lines 19~26). A *ind* with higher fitness is more likely to be selected during sampling. The crossover points are randomly generated. Based on experimental experience, the mutation probability p_m can be set to 0.0005~0.001.

Algorithm 4: Swapping Tensor Search based on GA

Input : *ModelInfo*(b), M_S , *iter*(iteration times), PS (population size), p_m (mutation probability)
Output: S

```

1  for  $i = 1$  to  $PS$  do
2       $ind_i \leftarrow \text{Zeros}(m)$ ;                                     //  $ind$  is initialized as a all-zero code
3       $ind_i \leftarrow \text{Mutate}(ind_i, i/PS)$ ;                     // Flip each bit of  $ind$  by probability  $i/PS$ 
4  end
5  for  $g = 1$  to iter do
6      for  $i = 1$  to  $PS$  do
7           $m_S \leftarrow \text{GetSwappingSize}(ind_i)$ ;
8          if  $m_S \geq M_S$  then
9               $t_i(b, ind_i) \leftarrow \text{TimeEvaluator}(ind_i)$ ;
10              $f_i \leftarrow \alpha t_i(b, ind_i) + \beta ((m_S - M_S)/M_S)$ ;
11          else
12               $f_i \leftarrow 0$ ;                                     // The total swapping memory size is not enough
13          end
14      end
15       $k, f_{temp} \leftarrow \max_{i \in [1, PS]} (f_i)$ ;                     // Get the best  $ind$  of the  $g$ th iteration
16      if  $f_{temp} > f_{best}$  then
17           $f_{best} \leftarrow f_{temp}$ ,  $ind_{best} \leftarrow ind_k$ ;
18      end
19      for  $i = 1$  to  $PS$  do
20           $p_i \leftarrow f_i / \sum_{j=1}^{PS} f_j$ ;
21           $ind_i \leftarrow \text{Sample}(p_i, ind)$ ;                     // Sample from all the old  $ind$  according to probability  $p_i$ 
22      end
23      for  $i = 1$  to  $PS/2$  do
24           $ind_{2i-1}, ind_{2i} \leftarrow \text{Cross}(ind_{2i-1}, ind_{2i}, c)$ ; // Swap the bits at indices 1 ~ 5 between  $ind_{2i-1}$  and  $ind_{2i}$ .
25           $ind_{2i-1} \leftarrow \text{Mutate}(ind_{2i-1}, p_m)$ ,  $ind_{2i} \leftarrow \text{Mutate}(ind_{2i}, p_m)$ ; // Flip each bit according to probability  $p_m$ 
26      end
27  end
28   $S \leftarrow \text{GetSwappingTensorSet}(ind_{best})$ ;

```

As shown in Algorithm 5, the TimeEvaluator evaluates the training time for each *ind* by accumulating computation and swapping time. In the forward propagation, the TimeEvaluator accumulates the forward computation time $t_F(layer.id, b)$ for each layer and records how many tensors are swapped out during that period (lines 7~10). If no free buffer exists, the waiting time for the swapping out of previous tensor is accumulated (lines 11~13). Similar to forward propagation, the TimeEvaluator accumulates the backward computation time $t_B(layer.id, b)$ and the waiting time for tensors to be swapped in during backward propagation (lines 16~27).

Algorithm 5: TimeEvaluator

```

Input : ModelInfo(b), CG, ind
Output: t(b, ind)
1 S ← GetSwappingTensorSet(ind), Sr ← Reverse(S); // Sr is used for swapping in
2 node ← the first node in CG, xs1 ← the first tensor in S, xs2 ← the first tensor in Sr;
3 while node ≠ the last node in computation_list do
4   layer ← GetLayerInfo(node);
5   tensors ← GetTensors(layer); // Get the feature map, weight and other relevant tensors of layer.
6   if forward propagation then
7     if free buffer exists then
8       t(b, ind) ← t(b, ind) +  $t_F(layer.id, b)$ ;
9       Check how many tensors in S were swapped out within  $t_F(layer.id, b)$ , xs1 ← the next tensor;
10      node ← the next node in CG; // Only when the computation is done
11    else
12      t(b, ind) ← t(b, ind) +  $size(x_{s1})/B$ ; // No free buffer, waiting for xs1 swapping out
13      xs1 ← the next tensor in S;
14    end
15  else
16    if tensors.position = gpu then
17      t(b, ind) ← t(b, ind) +  $t_B(layer.id, b)$ ;
18      Check how many tensors in Sr were swapped in within  $t_B(layer.id, b)$ , xs2 ← the next tensor to be swapped;
19      node ← the next node in computation_list; // Only when the computation is done
20    else
21      if tensors.position = cpu then
22        t(b, ind) ← t(b, ind) +  $size(x_{s2})/B$ ; // Waiting for xs2 swapping in
23        xs2 ← the next tensor in Sr;
24      else
25        t(b, ind) ← t(b, ind) + GetRecomputationTime(tensors); // xc.position = nodata
26      end
27    end
28  end
29  UpdateState(xc, xs1, xs2);
30 end

```

5 Evaluation

5.1 Experimental setup

Experimental platform: Our experimental platform is a CPU-GPU server equipped with two Intel Xeon 4215R CPUs (128GB memory) and one NVIDIA Tesla V100 GPU (32GB GPU memory). The CPU and GPU are connected via the PCIe 3.0x16 bus. The software environment includes the Ubuntu 18.04 operating system, CUDA 10.2 [22], and cuDNN 7 [23] software packages. The ATP framework is implemented based on a lightweight DNN training framework written in C++ [32]. The forward and backward propagation computations of all layers using the APIs provided in Nvidia’s cuDNN and cuBLAS libraries, which is consistent with popular deep learning frameworks such as PyTorch and TensorFlow.

Workloads: To comprehensively evaluate the effect of our approach on workloads of different depths and structures, we use multiple models including linear and nonlinear networks, convolutional neural networks

Table 4. Model structure of workloads

Workload	Types	Model structure			
		<i>Conv2_x</i>	<i>Conv3_x</i>	<i>Conv4_x</i>	<i>Conv5_x</i>
ResNet512	CNNs	3	8	156	3
ResNet1022	CNNs	3	8	326	3
RNN	RNNs	$(RNN\ layer + LN) \times 64$			
LSTM	RNNs	$(LSTM\ layer + LN) \times 64$			
GRU	RNNs	$(GRU\ layer + LN) \times 64$			
GPT	Transformer	$(ATTN + LN + FC + LN) \times 12$			

(CNNs), recurrent neural networks (RNNs), and transformer models as shown in Tab 4. The *Conv2_x* to *Conv5_x* denote the number four residual structures of ResNets [10]. The RNN, LSTM and GRU are respectively consist of the 3 types of recurrent layers and layer normalization layer (LN) [2, 7, 12]. The GPT has 12 Decoder structures, which including multi-head self-attention layer (SATTN) with residual and 12 attention heads, LN and fully connected layer (FC) [27]. Each batch of the input tensor for ResNet512 and ResNet1022 is RGB images with the default size of ImageNet ($3 \times 224 \times 224$) [8]. For RNN, LSTM and GRU, we use sequential data with sequence length 512. The input sample size of GPT is sequence length 512, beam size 1 and embedding size 768 [27]. All the tensors in our experiments are FP32 precision. We use throughput (samples/second) as the main evaluation metric in our experiments because it is a standardized indicator that reflects the speed of DNN training and is independent of the specific model training.

Baseline. We compare ATP with the following four baselines for tensor management in GPUs. We reimplemented these methods within our own framework by incorporating their tensor management strategies, primarily focusing on the selection of tensors for recomputation and swapping.

- **Normal:** Normal represents the original DNN model training without swapping and recomputation.
- **vDNN:** A swapping method based on expert experience [28]. Since the computation time of convolutional layer is usually longer than of other layer, vDNN swaps all the feature maps of convolutional layers for reaching a better overlap ratio between communication and computation time.
- **SwapAdviser:** SwapAdviser uses genetic algorithm to decides which tensors should be swapped or kepted in GPU memory for the shortest iteration time of DNN models [13]. In SwapAdviser, a swapping solution can be represented as a binary code, where each bit of the code uses 1 or 0 to denote a certain tensor needs to be swapped or not.
- **HOME:** HOME combines tensor recomputation and swapping to reduce memory footprint [11]. The search of tensor management solution is based on a customized particle swarm optimization (PSO) algorithm. Each particle denotes a solution, which can be represented as a sequence $[0, 1, 2, 2, 1, \dots]$. The value of 0, 1 and 2 in each bit denotes the corresponding feature map is keeping, swapping and recomputation, represently. For each particle, HOME attempts to swap each bit with the best particle in each iteration and determines whether to swap based on evaluating the time after the swapping of each bit. HOME iteratively updates the particle swarm in this manner to find the solution with the shortest model iteration time.

5.2 Training Performance

We first evaluate the training throughputs of different baselines on different workloads. As shown in Fig. 9, overall, ATP achieves larger batch sizes, and improves performance for most models compared to Normal training. The performance improvement brought by ATP is different for various models. For ResNet512 and ResNet1022, ATP improves the throughput by up to 1.02~1.12 $\times$, respectively. The throughput improvement of the other four networks is more obvious compared to the first two CNNs, reaching up to 1.49 $\times$ (Fig. 9f) that of Normal training.

Overall, ATP offers performance improvements across all workloads, but the gains are more significant for recurrent networks and GPT compared to CNNs. This disparity is primarily due to how changes in batch size

affect their computational intensity differently. The computations in recurrent networks and GPT are mainly driven by GEMM operations in recurrent and attention layers, whereas in CNNs, the dominant operations are convolutions. As seen in Table 1, the denominator of I_{GEMM} contains a constant term $4dk$, which is the product of the height and width of the weight matrix, while the constant term in I_{CONV} is $4c_o c_{rs}$, representing the product of the convolution kernel dimensions. Typically, the dimensions of convolution kernels in DNNs are much smaller than the dimensions of the weight matrices in GEMM. This implies that the increase in computational intensity with batch size is more significant for recurrent networks and GPT than for CNNs.

Compared to other methods, ATP achieves the highest throughput in the majority of models and batch sizes. For ResNet512 and ResNet1022, when the increment of batch size is small, the throughput of ATP is very close to that of HOME. However, as the further increase of batch size, the throughput of ATP shows obvious advantages, while the throughput of HOME shows a downward trend. The search algorithm of HOME simply swaps the bits between current solution and the historical best solution in each iteration without mutation of the bits of solution according to some probability, which makes the search of HOME easily fall into the local optimal solution. ATP flips the bits in its solution with a small probability in each iteration, enabling the algorithm to search for more possible policies. Since SwapAdviser only uses swapping, its throughput is limited by the significant communication overhead. ATP and SwapAdviser achieve almost the same performance on GPT because GPT is not suitable for recomputation, and ATP and SwapAdviser have similar methods for searching swapping solution. Except for CNNs, HOME can only slightly increase the batch size for other models because HOME focuses only on feature maps, while weights and other tensors have a larger proportion in RNNs and GPT. The solution of vDNN only swaps convolution feature maps, so the ability of vDNN to increase batch size is severely limited. ATP and SwapAdviser can reduce a lot of memory requirement for various workloads because their tensor management involves multiple types of tensors.

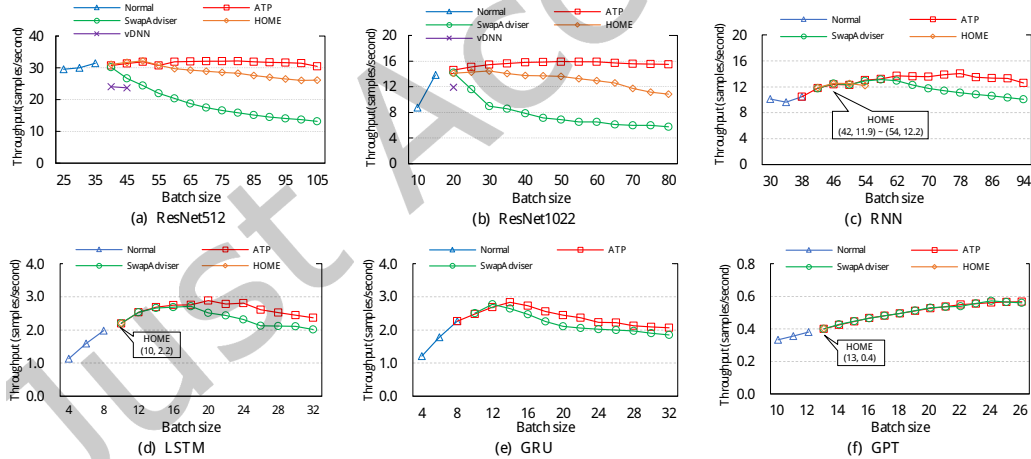


Fig. 9. Throughputs of methods with varying batch sizes for different DNN models.

5.3 Performance Breakdown Analysis

To analyze the influence of ATP and the baselines on model training in detail, we break down the whole training time into different parts. As shown in Fig. 10, we collect four parts of training time, where the swapping in/out time and the recomputing time denote the average total swapping and recomputing time during one training iteration, and the excess memory represents the memory required by training that exceeds the GPU memory limit.

The forward time and backward time in Fig. 10 represent the pure computation time of forward and backward propagation, excluding the recomputation time and synchronization time caused by tensor swapping. These two time values are the same for each method, as they are determined solely by the workloads. We take the batch size when the training throughputs of the 6 workloads reach their maximum, since these cases can effectively explain why ATP achieves the best performance. In general, the excess memory sizes of ATP and the baselines are very close, since these methods aim at the shortest training time and it's unnecessary to reduce more GPU memory. In Fig. 10a and Fig. 10b, the swapping time of ATP is shorter than that of HOME, but the recomputing time is slightly longer. It can be find that the swapping time of ATP is a bit shorter than the forward time, which means the total swapping time can be covered by forward computing time. With the same excess memory size, ATP reduces a significant swapping time by allocating more recomputation memory size, for example, in ResNet1022, the swapping time of ATP is shorter than HOME's by 0.67s and the recomputing time of ATP is only longer than that of HOME by 0.02s. This result shows that it is advantageous to allocate the total memory size of recomputation and swapping before searching specific tensor management solution. Since SwapAdviser only uses swapping, the swapping time is particularly long on all workloads and even dominates the whole training time. In Fig. 10c and Fig. 10d, ATP's swapping time can not be shorter that the forward time by reason that some of the tensors in the RNN and LSTM models can not be recomputed such as weights and some reserve spaces for RNN and LSTM layers. In these cases, the policies of ATP recomputes more tensors as much as possible to reduce the swapping time that is impossible to overlapped. The experimental results of ATP and SwapAdviser are similar in Fig. 10e, the swapping time of 44.6~44.7GB data can be completely covered by GPT's forward time so recomputation is useless for GPT.

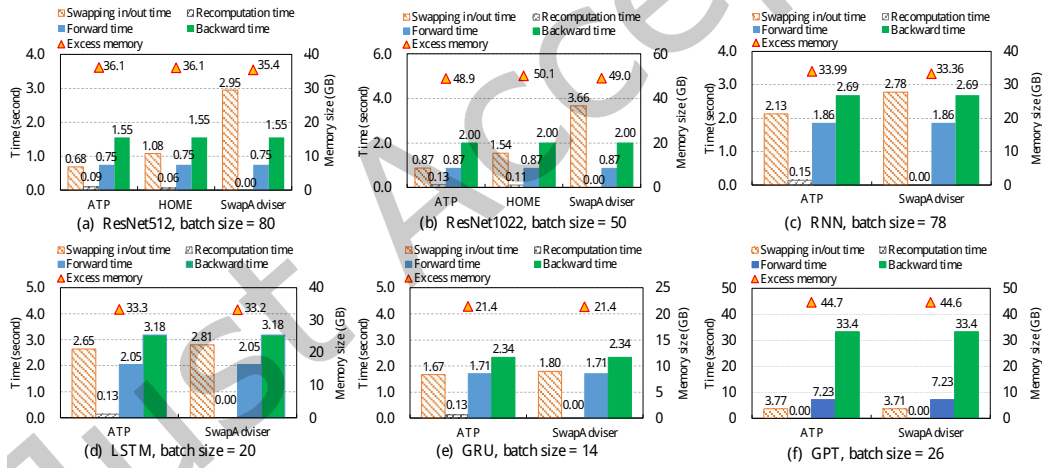


Fig. 10. Different parts of training time.

5.4 Throughput Peak Search Experiments

In Stage 1, ATP searches the batch size that can reach the throughput peak according to the predicted results of the throughput model from small to large. In this section, we compare the variation trend of the predicted throughput with the actual ATP throughput, as well as the error between them, to illustrate the feasibility of using the throughput model for searching. As shown in Fig. 11, the curves of ATP, TPM, and No limit represent the actual throughput, the throughput estimated by the throughput model and the ideal throughput without memory constraints (the ideal throughput is the ratio of batch size to the sum of computation times for all layers).

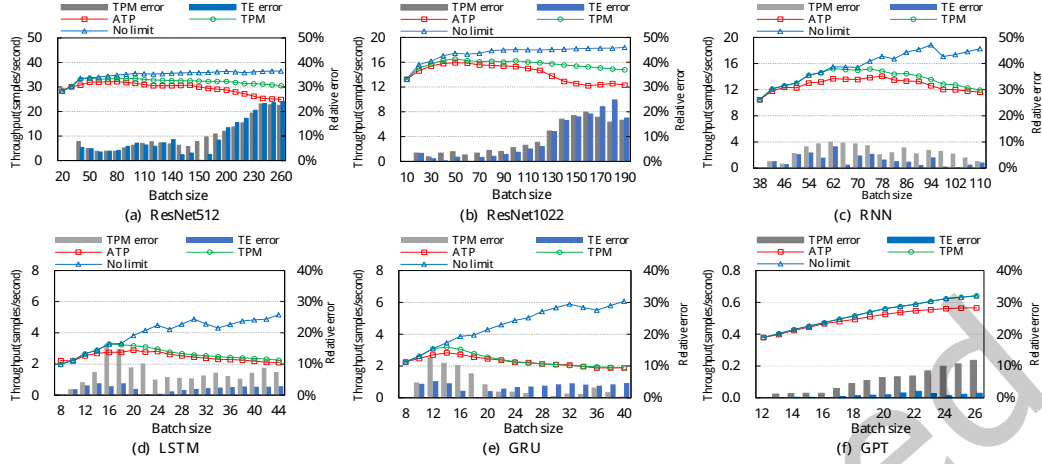


Fig. 11. Estimated throughputs of the throughput model and the TimeEvaluator

The throughput rate of No limit essentially increases with the enlargement of the batch size and tends to slow down when the batch size is large, as the utilization of CUDA cores approaches saturation at this point. The throughput model considers the continuously growing time overhead of recomputation and swapping, thus, the throughput shows an increasing-then-decreasing trend. This trend aligns with the speculation in Section 3.1. When the batch size is small, the TPM is close to the No limit, as the time overhead of recomputation and swapping is relatively small. As recomputation and swapping of GPU memory increase, TPM decreases after reaching its peak, deviating from the No limit. When the batch size approaches the limit, TPM's curve stabilizes, as the large GPU memory swapping essentially determines the throughput of model training. In Fig. 11f, TPM and No limit curves are essentially consistent because the computation time for each layer in GPT is very long, and even when the batch size approaches the limit, the time required for swapping does not exceed the computation time.

The TPM error in Fig. 11 represents the relative error between TPM throughput and actual throughput, which is generally less than 10%, reaching 20% in some cases. It is observed that for ResNet512, ResNet1022, and GPT models, TPM errors are larger when the batch size approaches the limit. The training time of these three models is always dominated by computation time. Increasing the batch size leads to more swapping tensors, and the swapping time for each tensor becomes longer. This increases the difficulty of fully covering communication time by computation time, leading to the actual throughput deviating from the theoretical value. But for RNN, LSTM and GRU, the larger TPM errors occur when the batch size is moderate, and the errors tend to decrease as the batch size continues to increase. When the batch size is very large, the training throughput is dominated by swapping time, and it is easy to cover computation time with swapping time. Therefore, the actual throughput is very close to the theoretical throughput. The TE error represents the error between the training time obtained by the TimerEvaluator during swapping tensor search and the actual training time. Since TimerEvaluator simulates the actual training process to obtain time, the TE error is generally smaller than the TPM error.

Despite some errors, as long as TPM and the actual throughput exhibit consistent trends, the requirements for throughput peak search can be satisfied. Table 5 shows the results of the throughput peak search. Except for ResNet512 and LSTM, ATP accurately locks the batch size corresponding to the peak throughput rate on other models. For ResNet512 and LSTM, the TPM errors lead to incorrect batch size selection. However, the throughput corresponding to the selected batch sizes obtain the errors of only 0.6% and 7.7% compared to the maximum throughput. The throughput is still improved compared to normal training.

Table 5. The result of Maximum throughput search

Workloads	ResNet512	ResNet1022	RNN	LSTM	GRU	GPT
Selected best batch size	50	50	74	16	14	26
Selected throughput	31.97	15.91	13.87	2.75	2.84	0.57
Practical best batch size	80	50	78	20	14	26
Practical max throughput	32.15	15.91	14.07	2.98	2.84	0.57

5.5 Memory Allocation

Fig. 12 illustrates the memory allocation of ATP for different workloads. For ResNet512 and ResNet1022, the memory size of recomputation is significantly larger than that of swapping. On the one hand, CNNs include nearly 2/3 of batch normalization layers and ReLU layers, where their memory requirement is similar to convolutional layers but with significantly less computation time. In addition to the batch normalization and activation layers, a considerable portion of the recomputation time for tensor in convolutional layers is less than the swapping time. Therefore, after allocating the memory that can overlap with both swapping and computation, ATP tends to allocate more recomputation memory. The situation is opposite for RNN, LSTM, and GRU models, where the swapping size exceeds the recomputation size. This is because the recurrent layers in these models account for the main memory overhead, and their recomputation overhead is greater than the swapping overhead. For GPT, almost no recomputation is used due to the long computation time.

The memory pool in Fig. 12 denotes the proportion of memory pool size to the required memory size for the whole model. In most workloads, the proportion of memory pool does not exceed 5% of the total memory requirement. However, in GPT training, the memory pool occupies 17%. This is mainly because the majority of the memory overhead in the GPT model comes from its 12 attention modules, and the tensor sizes in attention modules vary significantly. For example, the size of ReserveSpace size is approximately 100× that of Q, K or V matrix. To ensure that any tensor can utilize the memory pool, the buffer must be set to a large size.

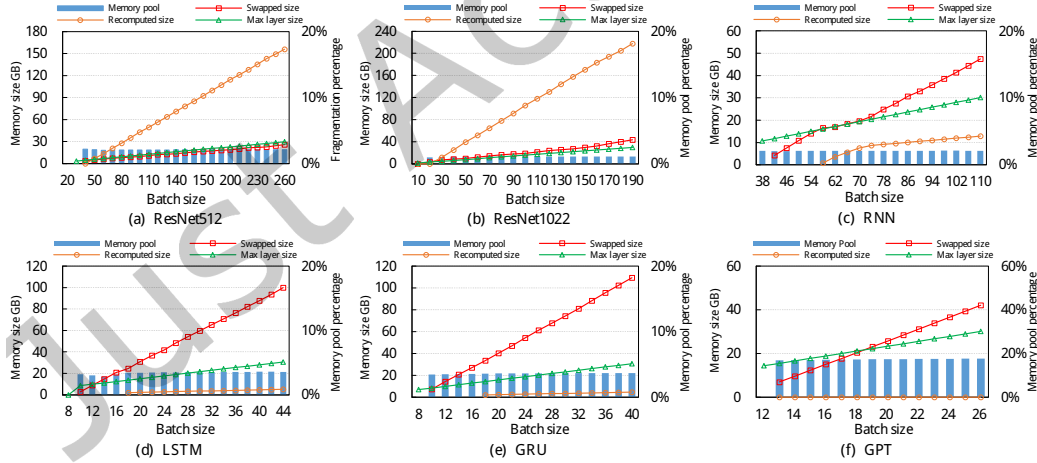


Fig. 12. Memory allocation for different workloads

Fig. 13 compares the average fragmentation size and the memory allocation time overhead per iteration during the training process between ATP's multi-buffer memory pool and the block-based memory pool (2MB Time, 512KB Time). ATP's memory pool significantly improves allocation speed. Its allocation time is only a fraction (tens to hundreds of times faster) of that of the block-based memory pool across all the workloads, as ATP only needs to update the buffer's state without searching for suitably sized and contiguous blocks. When the block

size is 512KB, the allocation time of the memory pool is significantly longer than when the block size is 2MB, due to the need to manage a larger number of blocks. In terms of fragmentation size, ATP's fragmentation size is generally positively correlated with batch size, as the buffer size in the multi-buffer memory pool is related to the tensor size. In the two ResNet models, ATP's fragmentation is relatively smaller compared to the block-based memory pool because the tensor sizes in the model are relatively uniform, and placing smaller tensors in the buffer does not cause significant fragmentation. However, in LSTM, GRU, and GPT, ATP's fragmentation is larger because these models contain tensors with significantly varying sizes, and each tensor is relatively large. ATP's buffers must ensure the placement of the largest tensor. Overall, ATP's memory pool has a significant performance advantage, but in terms of reducing fragmentation, it is more suitable for models with relatively uniform tensor sizes.

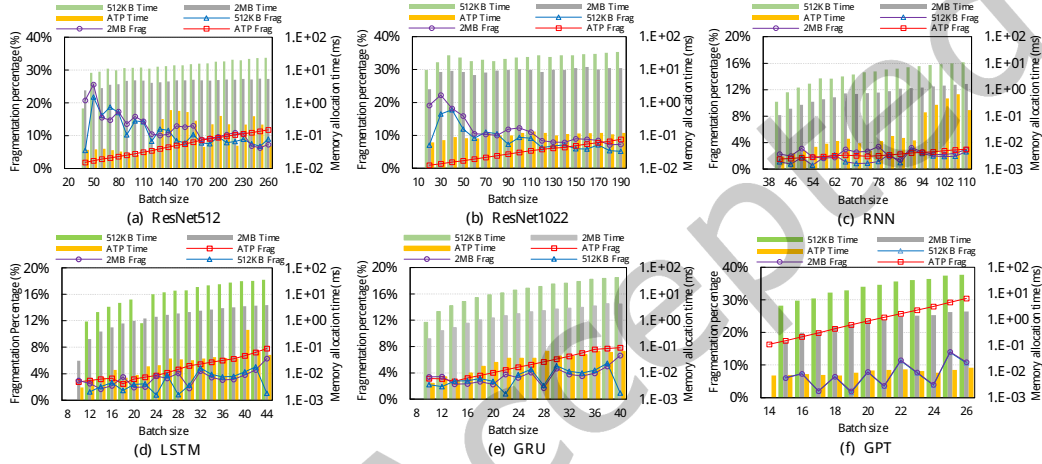


Fig. 13. The proportion of fragmentation in GPU Memory (32GB) and the time overhead of GPU memory allocation

5.6 The Search Efficiency of ATP

ATP conducts throughput peak search and tensor management solution search before DNN model training. In the throughput peak search, the step of batch size is set as 2 to 5, and ATP requires running one forward and backward propagation computation of the model after each batch size increment to collect model information until the throughput peak is found. The time for this process is determined by the computation time of the DNN model and is generally short, equivalent to the model training for only a few batches. Subsequently, ATP initiates the search for the tensor management solution.

Fig. 14 illustrates the convergence of tensor management solution search algorithms for ATP and other baseline methods. ATP and SwapAdviser generally achieve the optimal solution within 40 searches, while HOME typically converges in 1 to 4 searches due to its stringent criteria for selecting tensor management solution. HOME's PSO algorithm compares the encoding of each solution with the historical optimal solution bit by bit in each search, determining the value of each bit in the encoding by estimating the training time of DNN model. Although this approach converges quickly, it cannot generate new solution when trapped in a local optimum. Therefore, the curve for HOME remains unchanged after convergence. ATP and SwapAdviser use mutation operations to generate new solution in each search, making them less likely to get stuck in local optima.

Table 6 presents the time for each search. Overall, the search time for each algorithm is related to the number of layers in the DNN model because all three methods estimate the training time for each solution by simulating

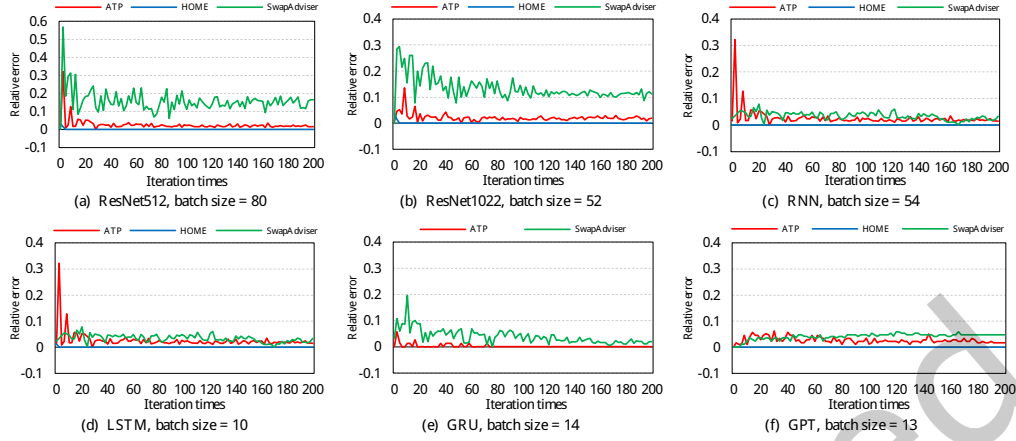


Fig. 14. The convergence versus search times, the y-axis represents the relative error in training time between the optimal solution of current search and the best historical solution, the particle swarm size for HOME and the population size for ATP and SwapAdviser are both set as 100.

Table 6. The average iteration time per search (Seconds)

Workloads	ResNet512	ResNet1022	RNN	LSTM	GRU	GPT
ATP	0.372	0.810	0.031	0.028	0.027	0.008
HOME	442	1853	2.43	3.344		0.275
SwapAdviser	0.454	1.1	0.028	0.029	0.032	0.008

the model training process. ATP and SwapAdviser have similar search time since they are both designed based on genetic algorithms. The search time for ATP is slightly less than SwapAdviser on the ResNet models because ATP first selects a large number of recomputed tensors based on tensor generation efficiency, resulting in a relatively smaller search space for swapping tensors. HOME has a significantly longer search time than other methods. In each iteration of its PSO, HOME compares all bits of each solution with the corresponding bit of the historical optimal solution and estimates the training iteration time. The time complexity of the PSO is $O(n^2)$ (particle swarm size $\times$ code size of solution). ATP perform only one time estimation for each solution after crossover and mutation, the time complexity is $O(n)$.

6 Discussion and Future Works

6.1 The Impact of ATP on Model Precision

The impact of ATP on model training includes the recomputation and swapping of tensors, as well as modifications to batch size. Swapping and recomputation are independent of data precision since each swapping operation involves transferring the entire tensor's data, and the recomputation operation regenerates the entire tensor's data with its original precision. Changes in batch size may affect model accuracy and the convergence process because the gradient used to update the weights is the mean of the gradients produced by all samples in a batch. In general, when batch size is limited by memory constraints, the impact of ATP on model training is positive. Firstly, the variance of gradients calculated from fewer samples is higher, causing larger fluctuations in model parameters and loss functions, which is also detrimental to GPU performance. Therefore, using ATP when batch size is small is beneficial. Secondly, ATP focuses solely on throughput because a simple increase in batch size can be fully achieved through gradient accumulation (as shown in Equation 11, where a batch of samples is divided into several sub-batches, and the gradients $\sum_{k=1}^{b_s} d_k w$ based on all sub-batch samples are accumulated

into gradient dw , which is used to update the weights to get gradient B). Combining gradient accumulation with ATP allows the model to be trained at the specified batch size while improving training speed. When using gradient accumulation, ATP effectively searches for the sub-batch size with the highest throughput. It can be seen from Equation 11, ATP increases the sub-batch size from b_s to b_{ATP} , and the number of sub-batches b/b_s is also reduced to b/b_{ATP} . By increasing throughput, ATP shortens the total time required to complete training of all sub-batches, while the gradient dw remains unchanged.

$$dw \equiv \frac{1}{b} \sum_{i=1}^b d_i w \equiv \frac{1}{b} \sum_{j=1}^{b/b_s} \sum_{k=1}^{b_s} d_k w \equiv \frac{1}{b} \sum_{j=1}^{b/b_{ATP}} \sum_{k=1}^{b_{ATP}} d_k w \quad (11)$$

6.2 Extending ATP to Multi-GPU Systems

While ATP is currently designed for single-GPU systems, it is desirable to support multi-GPU DNN training as well. We focus on how ATP should be extended to data parallel training in this section. With data parallelism, each GPU runs the same computation graph using a different sub-batch from the dataset. Consider two configurations of multi-GPU servers: one where each GPU is connected to a separate PCIe bus and accesses an independent NUMA node, and another where multiple GPUs share a PCIe bus.

Separate PCIe Bus: In this case, each GPU communicates with the CPU memory independently and can exclusively use the PCIe bandwidth. ATP can be directly applied to model training on each GPU by launching a separate ATP process for each GPU to control its recomputation and swapping.

Shared PCIe Bus: When GPUs share a PCIe bus, the tensor swapping of each GPU can only use a portion of the PCIe bandwidth. This requires modifications to the ATP throughput model and tensor selection algorithm. When the bandwidth B decreases, ATP will reduce the swapping size M_S and increase the recomputation size M_R according to Equation 8 and Algorithm 1. Assuming all GPUs in the system are of the same model and the total number is n , the bandwidth in Equations 5, 7, and 8 should be adjusted to B/n , allowing each GPU to use an equal share of bandwidth resources (i.e., the total swapping size per GPU remains equal). This is modified for load balancing since data parallel training requires an all-reduce operation on the parameters across all GPUs after completing their iterations of sub-batches. The multi-buffer memory pool design in Section 4.3 is independent of bandwidth and should be applied to each GPU directly. In Section 4.5, the increase in M_R leads to more recomputation tensors being selected. For swapping tensor selection and CUDA kernel control (Sections 4.6 and 4.4), applying the single-GPU method directly to multi-GPU systems is inappropriate, as simultaneous data transfers on the PCIe bus can cause additional delays due to bus contention. Even though theoretically each GPU can obtain a share of the bandwidth, the actual transfer time will exceed the theoretical value. Therefore, we should consider the tensor swapping of multiple GPUs as a whole, arranging the communication between each GPU and the host in sequence. The encoding *ind* in Equation 10 should expand from m bits to $m \times n$ bits to cover the swapping tensor set for each GPU, and the final swapping tensor sequence S should include tensors from all GPUs. The TimeEvaluator (Algorithm 5) in the objective function should extend to estimate the completion time for the last sub-batch across all GPUs. For CUDA kernel control, all swapping kernels should follow a unified launch sequence to avoid PCIe bus contention.

Model parallel training involves model partitioning and frequent communication between GPUs, which may consume a significant amount of PCIe bandwidth. Therefore, recomputation might be more suitable than swapping in this context. Currently, ATP does not have an adaptation method specifically for model parallelism.

6.3 Application to Recent Hardware and Neural Networks

In future work, we plan to extend the evaluation of ATP to the recent GPU hardwares and DNNs. By testing ATP on new GPUs with higher peak performance, larger memory capacity, and greater bandwidth, we aim to validate

and further enhance the scalability and applicability of our approach. Additionally, we will explore the impact of ATP on neural networks with larger parameter counts and more complex structures, ensuring that our method remains effective and relevant as DNNs and hardware continue to evolve.

7 Conclusions

This paper proposes ATP, a GPU memory management framework based on recomputation and swapping, aiming to achieve the throughput peak for DNN training by breaking the GPU memory limit. For this purpose, we propose a throughput model to assist ATP in theoretical throughput peak search for DNN training and the corresponding batch size, as well as the allocation for recomputation and swapping memory size. Additionally, we design a metric of tensor generation efficiency and search algorithms for specific recomputation and swapping tensors, and optimize the GPU memory pool and CUDA stream control, thereby improving ATP's actual training throughput to approach the theoretical peak.

The experimental results demonstrate that ATP can correctly search for the peak training throughput for various DNNs and its improvement in throughput is superior to other memory management methods in most cases. ATP achieves throughput improvements ranging from 1.02~1.49× across different DNNs and has the capability to expand batch sizes to the theoretical limit, enabling model training exceeding the GPU memory limit by 1.4~9.2×.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, and et al. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, Kimberly Keeton and Timothy Roscoe (Eds.). USENIX Association, 265–283.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer Normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *CoRR abs/1604.06174* (2016). [arXiv:1604.06174](http://arxiv.org/abs/1604.06174) <http://arxiv.org/abs/1604.06174>
- [4] Weiduo Chen, Xiaoshe Dong, Heng Chen, Qiang Wang, and et al. 2021. Performance evaluation of convolutional neural network on Tianhe-3 prototype. *J. Supercomput.* 77, 11 (2021), 12647–12665.
- [5] Xiaoming Chen, Danny Ziyi Chen, Yinhe Han, and Xiaobo Sharon Hu. 2018. moDNN: memory optimal deep neural network training on graphics processing units. *IEEE Transactions on Parallel and Distributed Systems* 30, 3 (2018), 646–661.
- [6] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. 2017. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282* (2017).
- [7] KyungHyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, and et al. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 1724–1734.
- [8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, 4171–4186.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Identity Mappings in Deep Residual Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 630–645.
- [11] Shuibing He, Ping Chen, Shuaiben Chen, Zheng Li, and et al. 2023. HOME: A Holistic GPU Memory Management Framework for Deep Learning. *IEEE Trans. Computers* 72, 3 (2023), 826–838.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. 1997. *Long Short-Term Memory*. Technical Report CS-TR-97-06. Institute of Computer Science, Faculty of Science, University of Freiburg, Germany.
- [13] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
- [14] Paras Jain, Ajay Jain, Aniruddha Nrusingha, Amir Gholami, and et al. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In *Proceedings of Machine Learning and Systems 2020, MLSys 2020, Austin, TX, USA, March 2-4, 2020*, Inderjit S. Dhillon,

- Dimitris S. Papailiopoulos, and Vivienne Sze (Eds.). mlsys.org. <https://proceedings.mlsys.org/book/320.pdf>
- [15] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, and et al. 2014. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*. ACM, 675–678.
 - [16] Jens Kehne, Jonathan Metter, and Frank Bellosa. 2015. GPUswap: Enabling Oversubscription of GPU Memory through Transparent Swapping. In *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, Istanbul, Turkey, March 14-15, 2015*, Ada Gavrilovska, Angela Demke Brown, and Bjarne Steensgaard (Eds.). ACM, 65–77.
 - [17] Jason Kennedy, Vishal Sharma, Blesson Varghese, and Carlos Reaño. 2023. Multi-Tier GPU Virtualization for Deep Learning in Cloud-Edge Systems. *IEEE Trans. Parallel Distributed Syst.* 34, 7 (2023), 2107–2123.
 - [18] Li Liu, Wanli Ouyang, Xiaogang Wang, Paul W. Fieguth, and et al. 2020. Deep Learning for Generic Object Detection: A Survey. *Int. J. Comput. Vis.* 128, 2 (2020), 261–318.
 - [19] Xinjian Long, Xiangyang Gong, Bo Zhang, and Huiyang Zhou. 2023. Deep learning based data prefetching in CPU-GPU unified virtual memory. *J. Parallel Distributed Comput.* 174 (2023), 19–31.
 - [20] Xinjian Long, Xiangyang Gong, Bo Zhang, and Huiyang Zhou. 2023. An Intelligent Framework for Oversubscription Management in CPU-GPU Unified Memory. *J. Grid Comput.* 21, 1 (2023), 11.
 - [21] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, and et al. 2018. Mixed Precision Training. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=r1gs9JgRZ>
 - [22] NVIDIA Corporation. 2019. *CUDA Toolkit*. NVIDIA Corporation. <https://developer.nvidia.com/cuda-toolkit>
 - [23] NVIDIA Corporation. 2020. *cuDNN: CUDA Deep Neural Network library*. NVIDIA Corporation. <https://developer.nvidia.com/cudnn>
 - [24] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, and et al. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2019, Madison, WI, USA, March 24-26, 2019*. IEEE, 304–315.
 - [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, and et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 8024–8035.
 - [26] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, and et al. 2020. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 891–905.
 - [27] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving Language Understanding by Generative Pretraining. *arXiv preprint arXiv:1801.06146* (2018).
 - [28] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and et al. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.
 - [29] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. 1986. Learning Representations by Back-Propagating Errors. *Nature* 323 (1986), 533–536.
 - [30] S. Suganyadevi, V. Seethalakshmi, and K. Balasamy. 2022. A review on deep learning in medical image analysis. *Int. J. Multim. Inf. Retr.* 11, 1 (2022), 19–38.
 - [31] Luming Sun, Shijin Gong, Tieying Zhang, Fuxin Jiang, and et al. 2023. SUFS: A Generic Storage Usage Forecasting Service Through Adaptive Ensemble Learning. In *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 3168–3181.
 - [32] Linnan Wang, Jinnian Ye, Yiyang Zhao, Wei Wu, and et al. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.
 - [33] Samuel Williams, Andrew Waterman, and David A. Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
 - [34] Fabian Wrede and Vincent von Hof. 2017. Enabling efficient use of algorithmic skeletons in cloud environments: container-based virtualization for hybrid CPU-GPU execution of data-parallel skeletons. In *Proceedings of the Symposium on Applied Computing, SAC 2017, Marrakech, Morocco, April 3-7, 2017*, Ahmed Seffah, Birgit Penzenstadler, Carina Alves, and Xin Peng (Eds.). ACM, 1593–1596.
 - [35] Su-Wei Yang, Zhao-Wei Qiu, and Ya-Shu Chen. 2020. GPU swap-aware scheduler: virtual memory management for GPU applications. In *SAC ’20: The 35th ACM/SIGAPP Symposium on Applied Computing, March 30 - April 3, 2020*. ACM, 1222–1227.
 - [36] Kaixin Zhang, Hongzhi Wang, Han Hu, Songling Zou, and et al. 2023. TENSILE: A Tensor Granularity Dynamic GPU Memory Scheduling Method Toward Multiple Dynamic Workloads System. *IEEE Trans. Knowl. Data Eng.* 35, 8 (2023), 8630–8643.

Received 20 February 2024; revised 3 September 2024; accepted 4 September 2024